

UNIVERSITÀ DEGLI STUDI DI TORINO
Dipartimento di Fisica

**REPORT
RETI NEURALI**

RICONOSCIMENTO DI SEGNALI STRADALI

A CURA DI
MARCO GORGONE

ANNO 2025/2026

Indice

1 Problema: Riconoscimento dei Segnali Stradali

In questa era dell'Intelligenza Artificiale, gli esseri umani stanno diventando sempre più dipendenti dalla tecnologia. Con l'evoluzione tecnologica, aziende multinazionali come Google, Tesla e molte altre stanno lavorando per automatizzare i veicoli, cercando di creare mezzi autonomi il più possibile precisi. Infatti, le auto a guida autonoma, in cui è il veicolo stesso a simulare il conducente, non necessitano di una guida umana. È lecito preoccuparsi degli aspetti legati alla sicurezza, in quanto è sempre presente la possibilità di incidenti; sappiamo infatti che nessuna macchina è ancora in grado di eguagliare la precisione degli esseri umani alla guida. I ricercatori stanno testando numerosi algoritmi per garantire una sicurezza stradale ottimale. Fra questi, un compito fondamentale è il **Riconoscimento dei Segnali Stradali**.

Quando si è in strada, si vedono vari segnali stradali come curve a sinistra o a destra, limiti di velocità, divieto di sorpasso ai veicoli pesanti, divieto di accesso, attraversamento pedonale per bambini, ecc., che è necessario seguire per una guida sicura. Allo stesso modo, i veicoli autonomi devono interpretare questi segnali e prendere decisioni per garantire la precisione. Per riconoscere a quale categoria appartiene un segnale stradale è quindi necessario classificarlo correttamente.

In questo lavoro viene proposto un modello di classificazione dei segnali stradali basato su una rete neurale convoluzionale (CNN).

In figura ?? è presente un esempio visivo che descrive il problema.

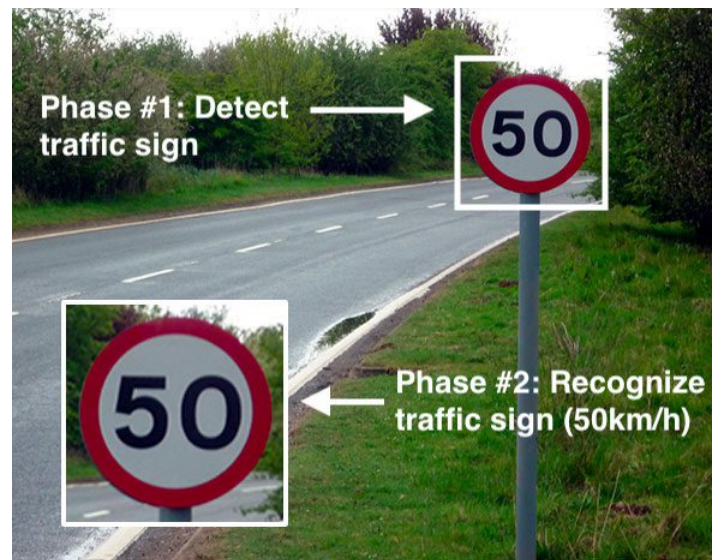


Figura 1: Esempio

1.1 Dataset

Il dataset completo delle immagini, scaricabile al link <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>, è composto da più di 50.000 foto di vari segnali stradali (limiti di velocità, attraversamenti, segnali stradali, ecc.), con una dimensione totale di circa 560 MB.

Per questo lavoro nello specifico sono state utilizzate le immagini contenute nel percorso DITS, con una dimensione di 450 MB.



Figura 2: Esempi visuali dei dati

Del dataset sono presenti due versioni una in cui i segnali sono divisi per segnale specifico e un'ultima da me scelta la divisione per macro categoria di segnale, in questo modo i segnali del dataset si dividono nelle loro 3 macrocategorie, a cui è poi stata aggiunta attraverso uno script una quarta categoria, arrivando ad avere le seguenti categorie:

- Divieto
- Pericolo
- Obbligo
- Background (utile per un secondo scopo)

2 Metodo

Verrà esaminato ora l'approccio eseguito per risolvere la problematica: il compito è configurato come un problema di classificazione con quattro categorie (indicazione, avvertimento, divieto, background).

2.1 Rete Utilizzata

In merito a ciò, dopo aver condotto alcuni esperimenti, i quali verranno descritti più dettagliatamente nel capitolo successivo, si è optato per l'implementazione di una Convolutional Neural Network (CNN) ispirata ad AlexNet (*LeNetColor*). A questa architettura, è stata applicata sia la tecnica della batch normalization che il dropout al fine di prevenire problemi di overflow durante il calcolo delle funzioni di attivazione e problemi di overfitting. Da adesso in poi il modello verrà nominato come **MiniAlexNetV2**. Nella figura ?? è possibile osservare una rappresentazione schematica della rete LenetColor che è il punto di partenza e la prima modifica eseguita per ottenere la rete MiniAlexNet, che sono state entrambe il punto di partenza per poi arrivare alla rete MiniAlexNetV2 in figura ??, la quale è stata poi scelta come rete finale.

Negli input della rete, sono state adottate le cosiddette "regole generali approssimative": i batch di dati sono stati configurati con dimensioni di 1024 per la fase di addestramento e di 2048 per le fasi di convalida e test, implementando MaxPooling con dimensione 2 e uno stride di dimensione 1. Come funzione di attivazione, è stata impiegata l'Unità Lineare Rettificata, nota come ReLU, la quale restituisce il valore stesso della combinazione se i dati sono positivi, mentre annulla quelli negativi. Tutti questi parametri sono formalmente espressi mediante un'apposita equazione matematica (??):

$$f(x) = \begin{cases} 0 & \text{se } x < 0 \\ x & \text{se } x > 0 \end{cases} \quad (1)$$

2.2 Funzione di Loss

La funzione di Loss utilizzata è la *Cross Entropy Loss* descritta in (??):

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \cdot \log(\hat{y}_{ij}) \quad (2)$$

dove:

N è il numero di campioni,

C è il numero di classi,

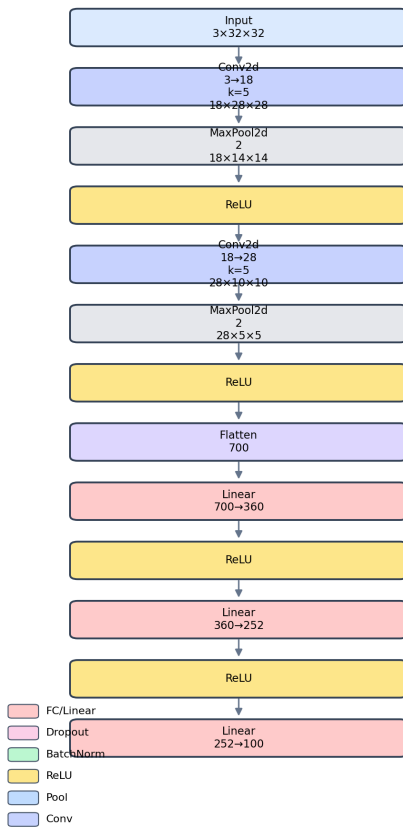
y_{ij} è la vera probabilità della classe j per il campione i ,

$\hat{y}_{ij}$ è la probabilità predetta della classe j per il campione i .

La formula della cross entropy loss, la quale consente il confronto tra due distribuzioni di probabilità, risulta fondamentale in quanto la rete **MiniAlexNetV2** fornisce per ciascuna immagine una probabilità associata all'appartenenza a una specifica classe. È pertanto necessario effettuare un confronto tra questa distribuzione di probabilità

LeNetColor

Schema architetturale verticale



MiniAlexNet

Schema architetturale verticale

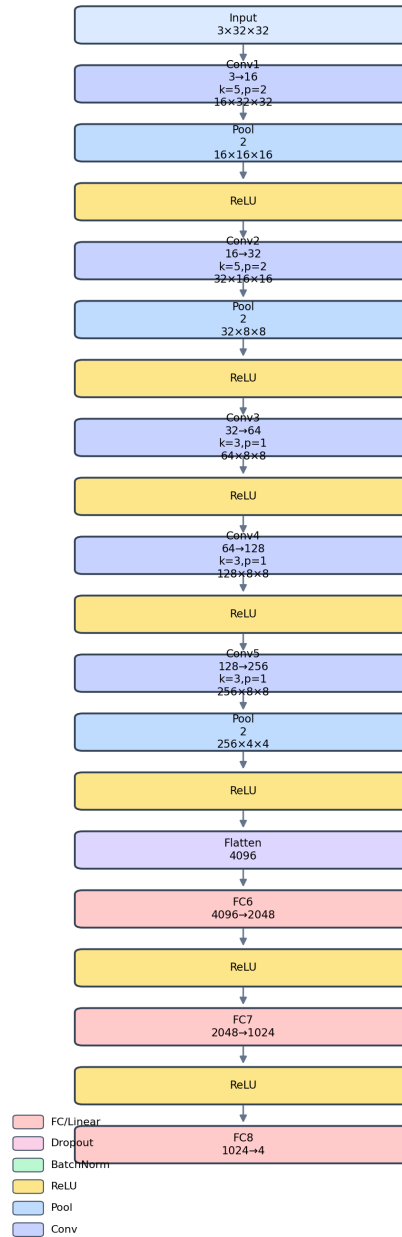


Figura 3: Confronto tra le tre reti neurali

MiniAlexNetV2

Schema architetturale verticale

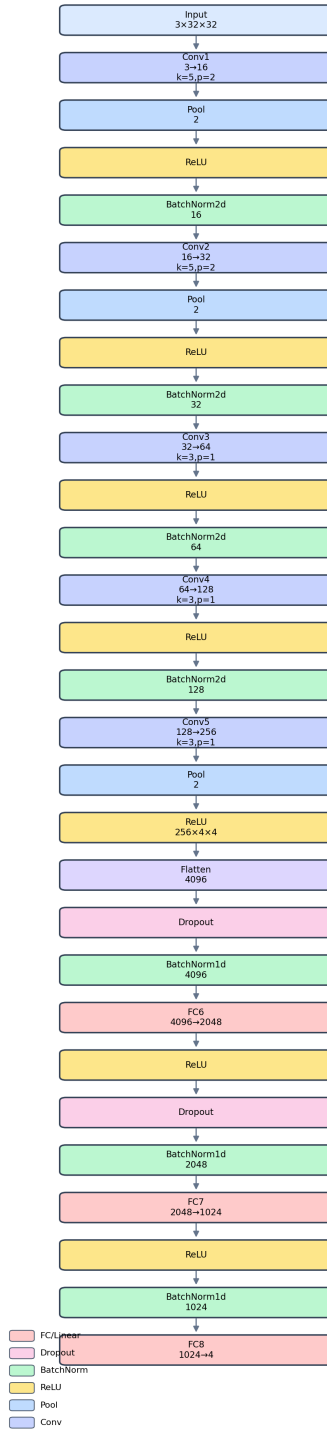


Figura 4: MiniAlexNetV2, rete utilizzata per il training

predetta e la distribuzione "reale", la quale assume un valore diverso da zero solamente per la classe alla quale l'immagine in input appartiene.

In termini formali, la funzione di Loss che è stata utilizzata presenta una leggera variazione rispetto a quella precedentemente menzionata. L'equazione adottata è la (??):

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_i^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)) \quad (3)$$

dove:

N è il numero di classi,

y_i è la vera probabilità della classe i ,

$\hat{y}_i$ è la probabilità predetta della classe i .

2.3 Valutazione

La validità delle previsioni è stata valutata attraverso il parametro di accuratezza. Tale parametro è stato derivato definendo una matrice di confusione elementare, come indicato nella tabella ??:

	Realtà	
	Positivo	Negativo
Previsioni Positivo	Vero Positivo	Falso Positivo
Previsioni Negativo	Falso Negativo	Vero Negativo

Tabella 1: Matrice di Confusione

Da questa analisi, è possibile definire l'accuratezza del modello come il rapporto tra il numero di previsioni corrette e il numero totale di previsioni, come indicato nell'equazione (??):

$$\text{Accuracy} = \frac{\text{Numero di Previsioni Corrette}}{\text{Numero Totale di Previsioni}} \quad (4)$$

Questo parametro fornisce una misura della corretta classificazione effettuata dal modello rispetto al numero complessivo di previsioni eseguite.

Oltre all'accuratezza, è stata elaborata anche la curva ROC, una curva grafica che illustra le prestazioni di un modello di classificazione al variare della soglia di decisione.

3 Analisi

Inizialmente sono state condotte una serie di analisi con l'obiettivo principale di individuare il modello più promettente. Successivamente, ci si è dedicati all'indagine delle configurazioni più adatte del momento, al fine di massimizzare l'accuratezza del sistema.

3.1 Scelta del Modello

Per identificare il modello ottimale, sono stati eseguiti diversi approfondimenti. Nel primo esperimento è stata impiegata una rete nota come **AlexNet**; successivamente è stata considerata una rete basata su essa, denominata **MiniAlexNet**; infine, introducendo sia layer di Dropout sia layer di Batch Normalization, è stata ottenuta la versione **MiniAlexNetV2**.

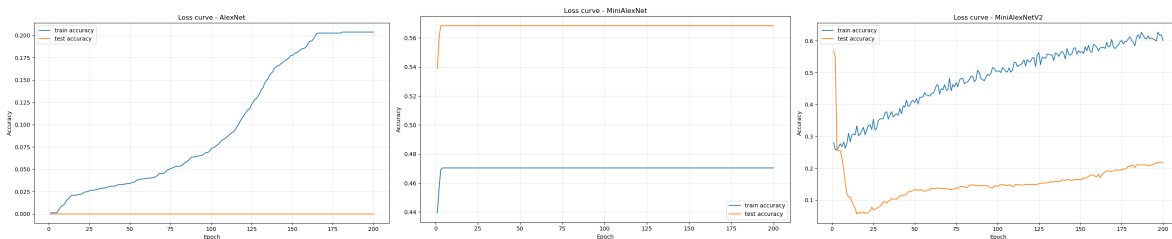


Figura 5: Confronto tra le curve di accuratezza delle tre reti

L'esame della Figura ?? evidenzia in modo chiaro come la rete **MiniAlexNetV2** raggiunga un'accuratezza sensibilmente superiore rispetto alle altre due configurazioni. In particolare, la **MiniAlexNet** mostra un andamento poco stabile e complessivamente modesto, mentre la rete **AlexNet** presenta un comportamento migliore rispetto alla MiniAlexNet, pur rimanendo inferiore alla MiniAlexNetV2. Pertanto, dal punto di vista dell'accuratezza, la rete MiniAlexNetV2 emerge come la soluzione più efficace.

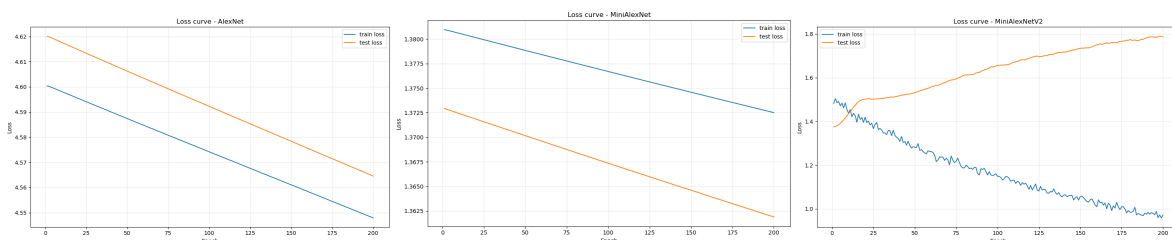


Figura 6: Confronto tra le curve di loss delle tre reti

Dall'analisi della Figura ?? emergono differenze significative tra le tre reti, tutte addestrate per 200 epoche. La rete **MiniAlexNetV2** mostra valori di loss più bassi rispetto alle altre architetture e una dinamica di apprendimento più efficace. Tuttavia, durante la fase di test, la curva evidenzia un iniziale aumento della loss, fenomeno compatibile con una fase di instabilità nelle prime epoche di addestramento; successivamente, la curva tende a stabilizzarsi su valori più contenuti.

La rete **MiniAlexNet**, pur presentando un comportamento più regolare rispetto ad AlexNet, mostra comunque prestazioni limitate e una capacità di generalizzazione contenuta. La rete **AlexNet**, infine, evidenzia un miglioramento lieve ma complessivamente inferiore rispetto alla MiniAlexNetV2.

Nel complesso, questi risultati hanno portato a concentrare gli sforzi sull'ulteriore sviluppo della rete **MiniAlexNetV2**.

I parametri utilizzati per queste prove sono stati:

- Tasso di apprendimento (*Learning Rate*): 0.0001
- Momento (*Momentum*): 0.6

3.2 ROC Curve

Una volta individuate le tre configurazioni più significative, il processo di training è stato reiterato per ulteriori 200 epoche e successivamente sono state calcolate le rispettive ROC curve, riportate in Figura ???. La curva ROC, che descrive la relazione tra il tasso di veri positivi (TPR) e il tasso di falsi positivi (FPR) al variare della soglia di classificazione, è stata utilizzata come ulteriore criterio di valutazione per confrontare le capacità discriminanti dei tre modelli.

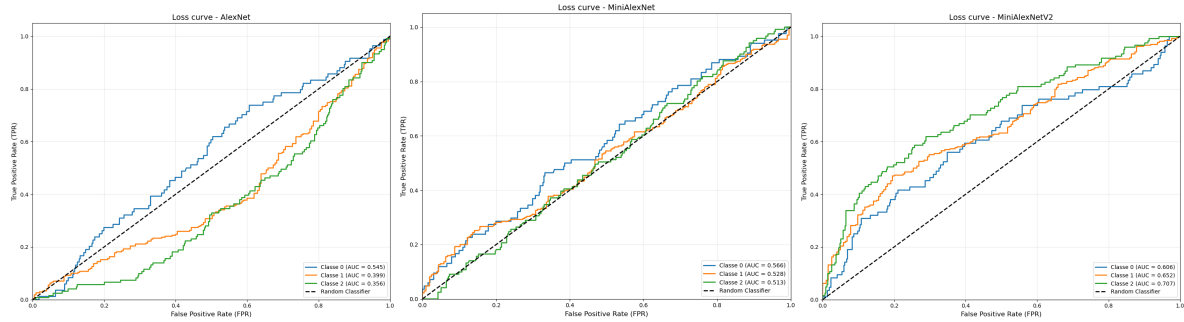


Figura 7: Confronto tra le ROC curve delle tre reti

Dall'analisi delle curve ROC in Figura ??? risulta evidente che la rete **MiniAlexNetV2** è quella con le prestazioni migliori. In particolare, i valori di AUC ottenuti sono pari a 0.606 per la classe 0, 0.652 per la classe 1 e 0.707 per la classe 2, evidenziando una capacità discriminante superiore rispetto alle altre reti. La rete **MiniAlexNet** presenta invece risultati prossimi al caso casuale, con AUC pari a 0.566, 0.528 e 0.513 rispettivamente per le tre classi. La rete **AlexNet** mostra infine le prestazioni peggiori, con AUC pari a 0.545 per la classe 0, 0.399 per la classe 1 e 0.356 per la classe 2.

Sulla base del confronto tra le tre curve ROC, la rete **MiniAlexNetV2** è stata quindi selezionata come modello più promettente, in quanto mostra il miglior compromesso tra capacità discriminante e stabilità delle prestazioni.

4 Codice

Il codice da me utilizzato è stato suddiviso e strutturato in modo tale da avere all'interno della sotto cartella "python_file" i seguenti file:

- Il file *dataclass* definisce le classi e le trasformazioni utilizzate per la gestione del dataset di segnali stradali. In particolare, include le classi per il training (*StreetSignDataset*) e per il testing (*StreetSignTest*), oltre a trasformazioni quali il ridimensionamento (*Rescale*), il ritaglio casuale (*RandomCrop*), il ritaglio specifico (*Crop*) e la conversione in tensori (*ToTensor*). È inoltre presente la funzione *show_landmarks_batch*, impiegata per visualizzare batch di immagini con i relativi landmarks. Nel complesso, il file fornisce l'infrastruttura necessaria per la preparazione dei dati.
- Il file *function* raccoglie le funzioni principali per l'addestramento e il test del classificatore. La prima parte del codice esegue il training del modello mediante *Stochastic Gradient Descent* e *Cross Entropy Loss*, con il supporto di TensorBoard e del salvataggio dei pesi. La seconda parte valuta il modello sul set di test, restituendo le predizioni e le etichette reali. Inoltre, nello stesso file sono presenti le funzioni per salvare i risultati in formato JSON e CSV e per generare i grafici delle curve di accuratezza e di loss. È infine inclusa una funzione per il calcolo e la visualizzazione della curva ROC a partire dalle label e dalle predizioni del modello.
- Il file *network* implementa le architetture delle reti neurali testate, includendo diverse varianti e l'impiego di tecniche come *Batch Normalization* e *Dropout* per migliorare le prestazioni e la capacità di generalizzazione del modello.
- Il file *detect_and_classify* implementa la procedura utilizzata per applicare YOLO al riconoscimento dei segnali stradali all'interno di immagini acquisite in un contesto reale. In questa fase, l'immagine viene inizialmente analizzata da YOLO, addestrato su immagini reali, per individuare il segnale stradale; successivamente, la porzione ritagliata viene fornita alla rete **MiniAlexNetV2** per determinarne la classe. In questo modo, il sistema realizza una pipeline completa di detection e classification.

Infine, sono stati creati alcuni notebook in Jupyter, nei quali è stata utilizzata la struttura dati definita in precedenza insieme alle funzioni di training e test, al fine di addestrare il modello. Prima dell'addestramento, le immagini vengono convertite nel formato richiesto dalla struttura dati e sottoposte alle opportune trasformazioni. I notebook realizzati sono i seguenti:

- *train*, che si occupa dell'addestramento delle reti *AlexNet*, *MiniAlexNet* e *MiniAlexNetV2*. In particolare, il notebook esegue il caricamento delle immagini, la loro conversione in tensori e l'addestramento dei modelli, restituendo infine il valore di accuratezza ottenuto da ciascuna rete.
- *result*, che carica i tre modelli precedentemente addestrati ed esegue una previsione a partire da un'immagine fornita tramite URL oppure da un'immagine

locale. Il notebook restituisce quindi la classe predetta del segnale stradale, tra cui *indicatory* (indicazione), *prohibitory* (divieto) e *warning* (pericolo). In caso di errore nella previsione, viene visualizzato il messaggio *ERROR 404 NOT FOUND*. La classe *Background*, sebbene presente nella fase di addestramento, non viene considerata in quanto non rilevante ai fini dell'applicazione.

- *test_reale*, che carica esclusivamente il modello *MiniAlexNetV2* e lo utilizza per eseguire la previsione della categoria del segnale stradale presente in un'immagine reale. In questo caso, l'immagine viene prima elaborata per individuare e ritagliare il segnale stradale, che viene successivamente passato alla rete *MiniAlexNetV2* per la classificazione. La classe predetta viene quindi sovrapposta all'immagine, che viene infine visualizzata.

4.1 Caso Reale

Analizziamo più in dettaglio il notebook *test_reale* in cui è presente la parte di codice relativa al test della rete neurale in un contesto urbano. Se, per esempio, dotassimo un veicolo di una telecamera per riprendere la strada in tempo reale, potremmo acquisire delle immagini durante il tragitto, e verificare la presenza di segnali stradali inviando infine l'output al sistema di bordo o al guidatore. Per fare ciò, ci si è posti il problema di come isolare la singola immagine del segnale rispetto al contesto; per eseguire questa ricerca sono stati studiati due processi:

- **Divisione in sottoimmagini:** consiste nel suddividere l'immagine in numerose porzioni e verificare per ognuna di esse se contiene un segnale stradale;
- **Uso di YOLO:** utilizzo di uno strumento di object detection che permette di individuare le regioni d'interesse (ROI) all'interno delle immagini, identificando le porzioni candidate alla classificazione;

4.1.1 Divisione

L'uso della divisione in sottoimmagini è la soluzione più semplice, ma ha presentato scarsi risultati sia in termini di prestazioni che di affidabilità. Il motivo risiede nella necessità di processare sequenzialmente ogni ritaglio, causando lentezza di calcolo. Inoltre, l'accuratezza è limitata dal fatto che il dataset di addestramento non copre tutte le possibili variazioni del contesto circostante, in più si ha il problema che quando noi eseguiamo il taglio dell'immagine non sempre il segnale ricadrà sul centro della sottoimmagine ma ve ne potrebbe essere solo una parte, questo fa sì che la rete non è in grado di riconoscere il segnale o di creare dei falsi positivi.

4.1.2 YOLO

Per ovviare ai limiti della suddivisione manuale, nel notebook *test_reale* è stato implementato l'uso dell'algoritmo **YOLO** (You Only Look Once). A differenza dell'approccio precedente, YOLO esegue l'object detection in un unico passaggio, identificando istantaneamente le coordinate dei segnali stradali (*Bounding Boxes*) all'interno di scene complesse, per poi passare la porzione d'immagine ritagliata col segnale alla rete *MiniAlexNetV2* per poter identificare la tipologia di segnale stradale.

Il flusso di lavoro implementato prevede i seguenti passaggi:

1. Caricamento dell'immagine ad alta risoluzione catturata nel contesto urbano.
2. Applicazione del modello YOLO per individuare le regioni d'interesse (ROI) corrispondenti ai segnali.
3. Ritaglio (*cropping*) automatico delle aree individuate.
4. Passaggio dei ritagli alla rete **MiniAlexNetV2** per la classificazione finale della macro-categoria.

Questo approccio ibrido ha permesso di ottenere una precisione estremamente elevata, poiché la rete di classificazione lavora solo su porzioni di immagine già identificate come "segnali", eliminando il rumore di fondo del contesto cittadino e riducendo drasticamente i tempi di calcolo rispetto alla scansione sequenziale.

5 Conclusione

Le conclusioni raggiunte suggeriscono che il modello proposto è in grado di affrontare con successo il compito assegnato, con possibilità di ulteriori miglioramenti. A questo proposito, si suggerisce di:

- Aumentare la quantità e la diversità dei dati di addestramento: utilizzare un set di dati più ampio e vario per migliorare la capacità di generalizzazione della rete.
- Sperimentare con diverse architetture di reti neurali: testare modelli con architetture diverse per identificare la più adatta al compito di riconoscimento dei segnali stradali.
- Utilizzare tecniche di regolarizzazione per ridurre l'*overfitting*: implementare tecniche di regolarizzazione per evitare che la rete si adatti eccessivamente ai dati di addestramento.
- Ottimizzare gli *hyperparametri*: regolare parametri come il tasso di apprendimento e la dimensione del batch per massimizzare le prestazioni del modello.
- Esplorare la quantizzazione dei pesi: ridurre la complessità della rete attraverso la quantizzazione dei pesi, mantenendo comunque un livello accettabile di accuratezza.
- Analizzare le attivazioni nei vari strati della rete: approfondire la comprensione del processo di apprendimento esaminando le attivazioni nei vari strati della rete neurale.

Per riassumere, questa esperienza non rappresenta solo un punto di arrivo, piuttosto un punto di partenza per ulteriori esplorazioni nel campo del riconoscimento dei segnali stradali attraverso l'impiego di modelli di machine learning. L'apertura a nuove idee, l'iterazione continua e la volontà di esplorare nuovi approcci saranno fondamentali per affrontare le sfide emergenti e migliorare costantemente le prestazioni dei modelli proposti. Un prossimo possibile caso d'uso potrebbe essere non identificare più la macrocategoria dei segnali ma andare nel dettaglio sul singolo segnale.

Appendice

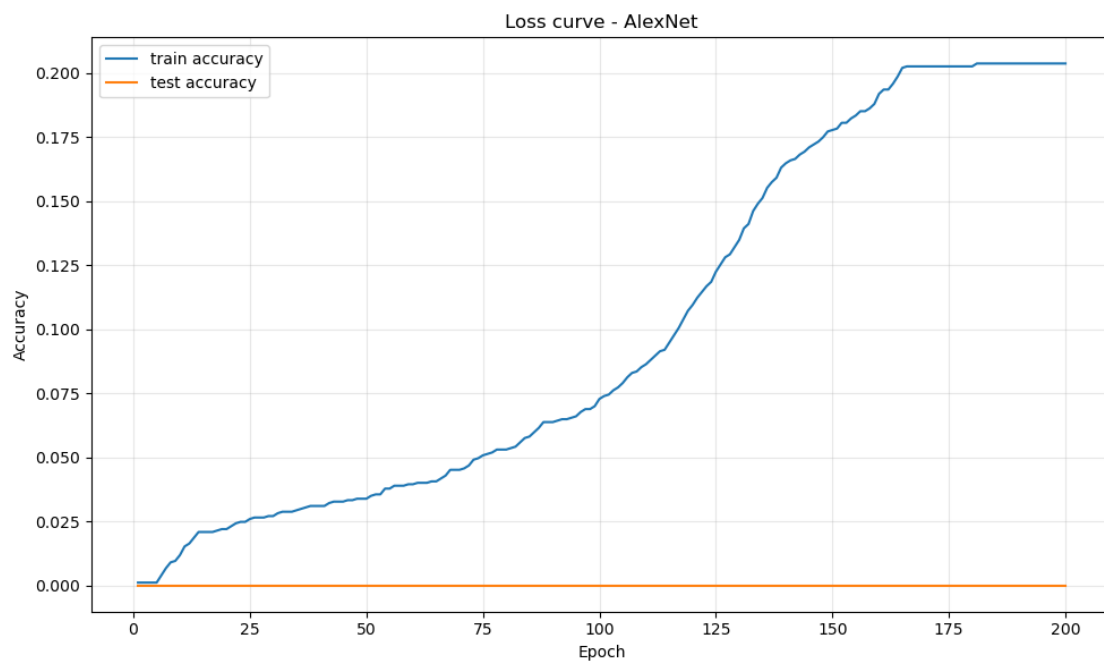


Figura 8: Curva di accuratezza della rete AlexNet.

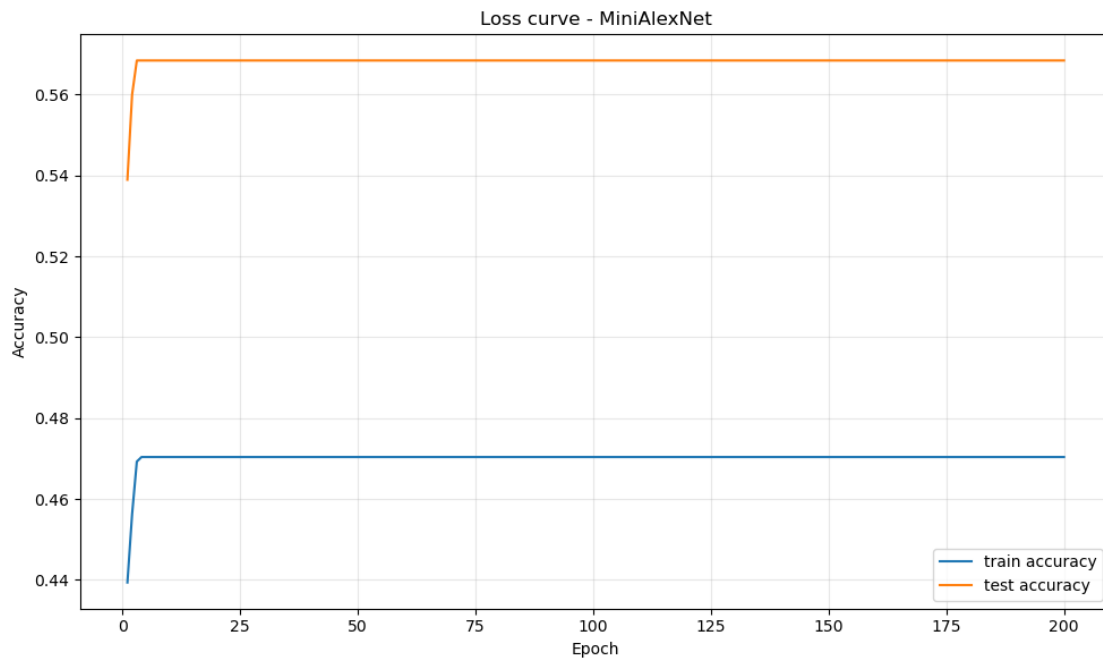


Figura 9: Curva di accuratezza della rete MiniAlexNet.

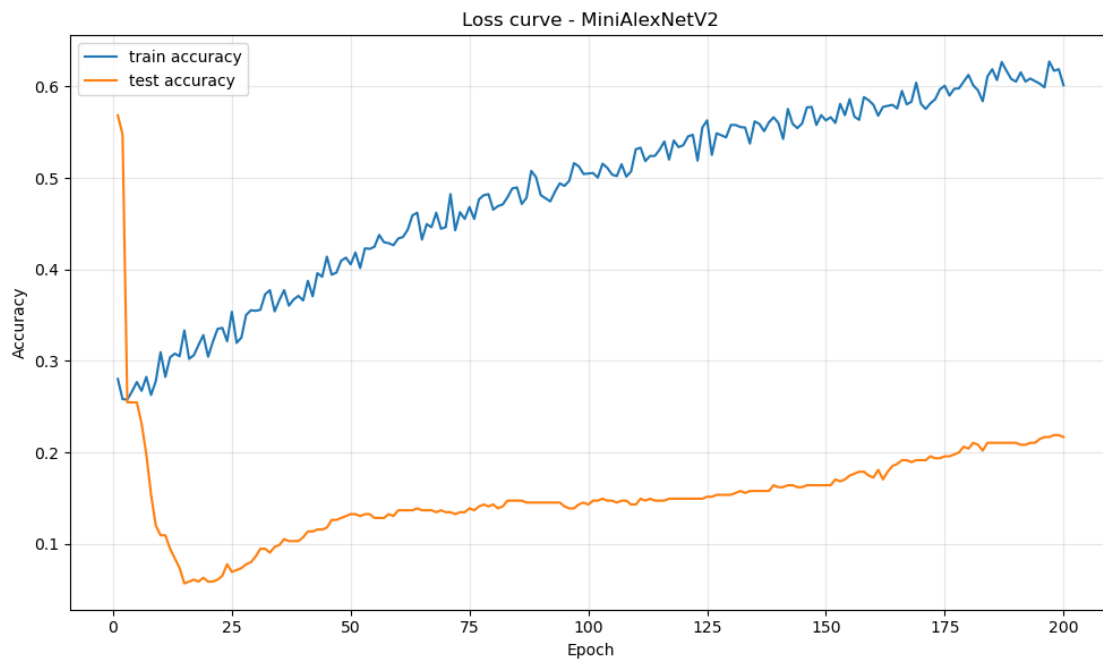


Figura 10: Curva di accuratezza della rete MiniAlexNetV2.

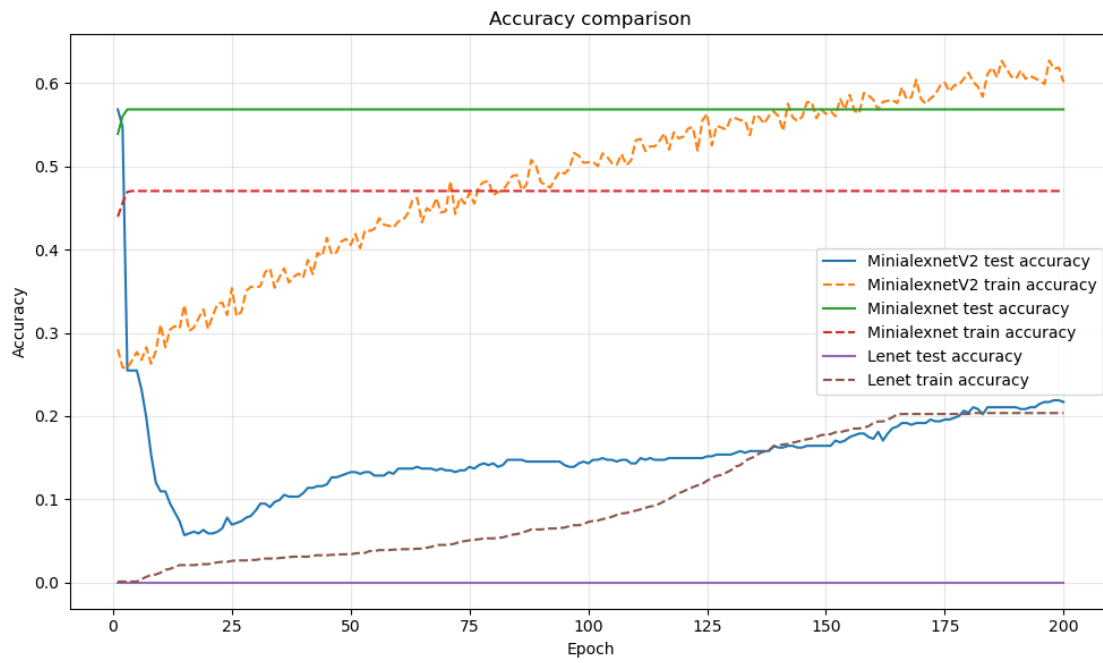


Figura 11: Curve di accuratezza delle reti.

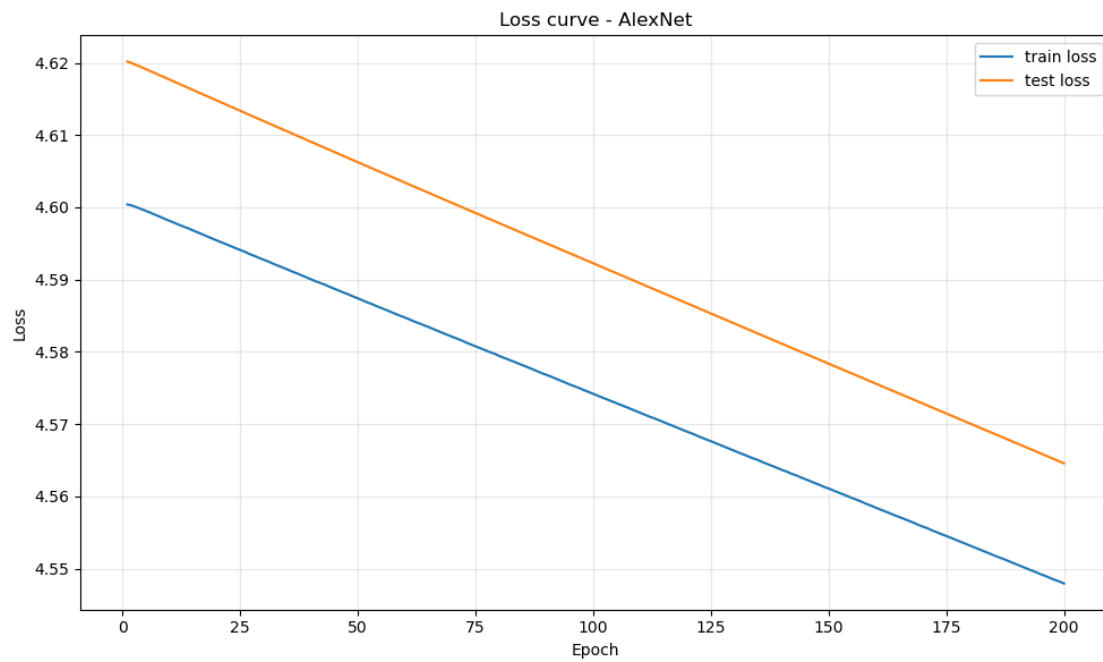


Figura 12: Curva di loss della rete AlexNet.

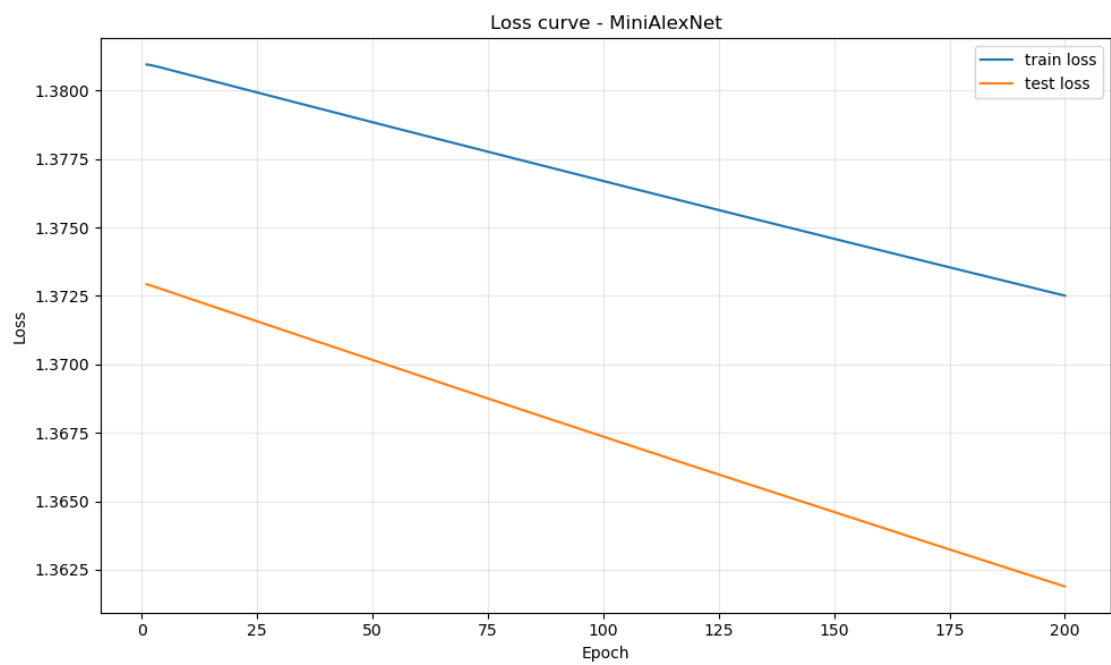


Figura 13: Curva di loss della rete MiniAlexNet.

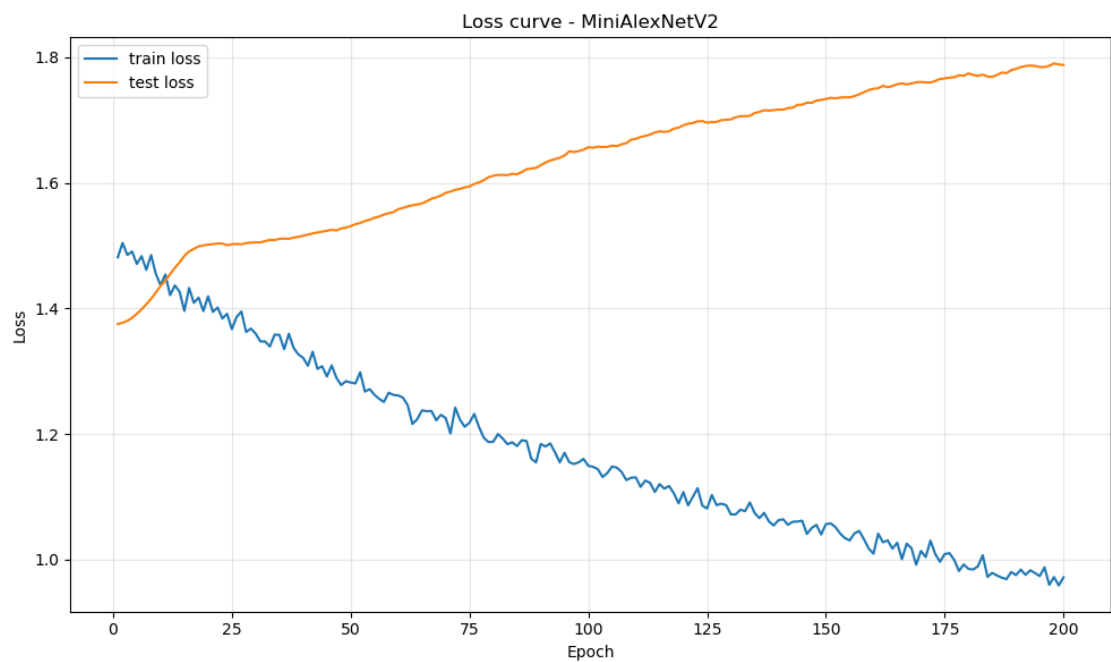


Figura 14: Curva di loss della rete MiniAlexNetV2.

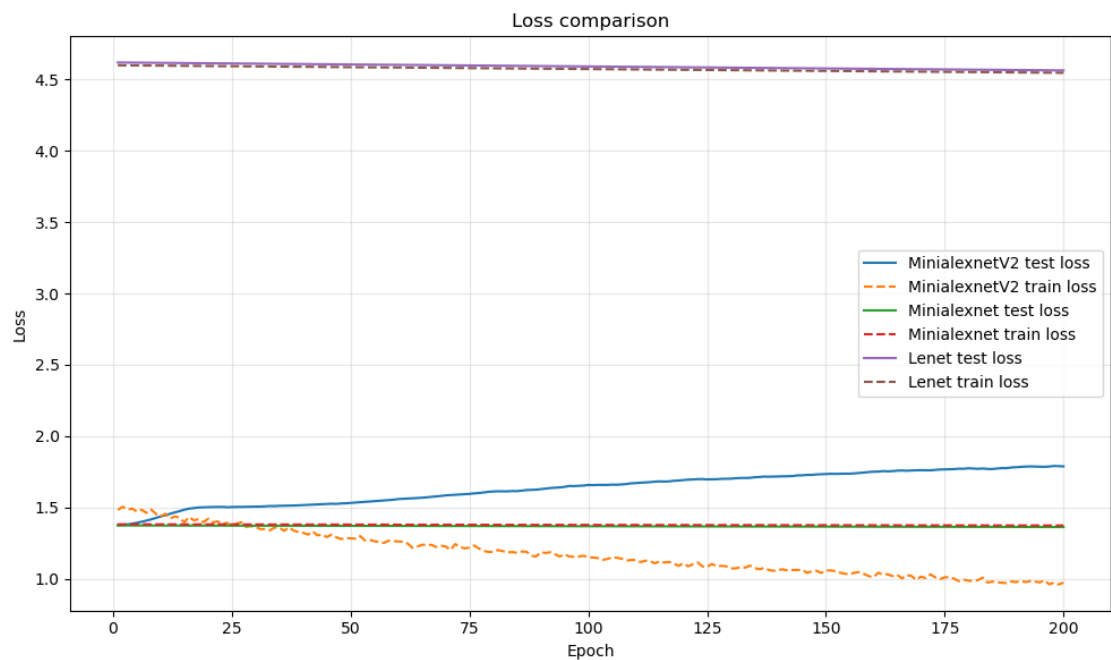


Figura 15: Curve di loss delle reti.

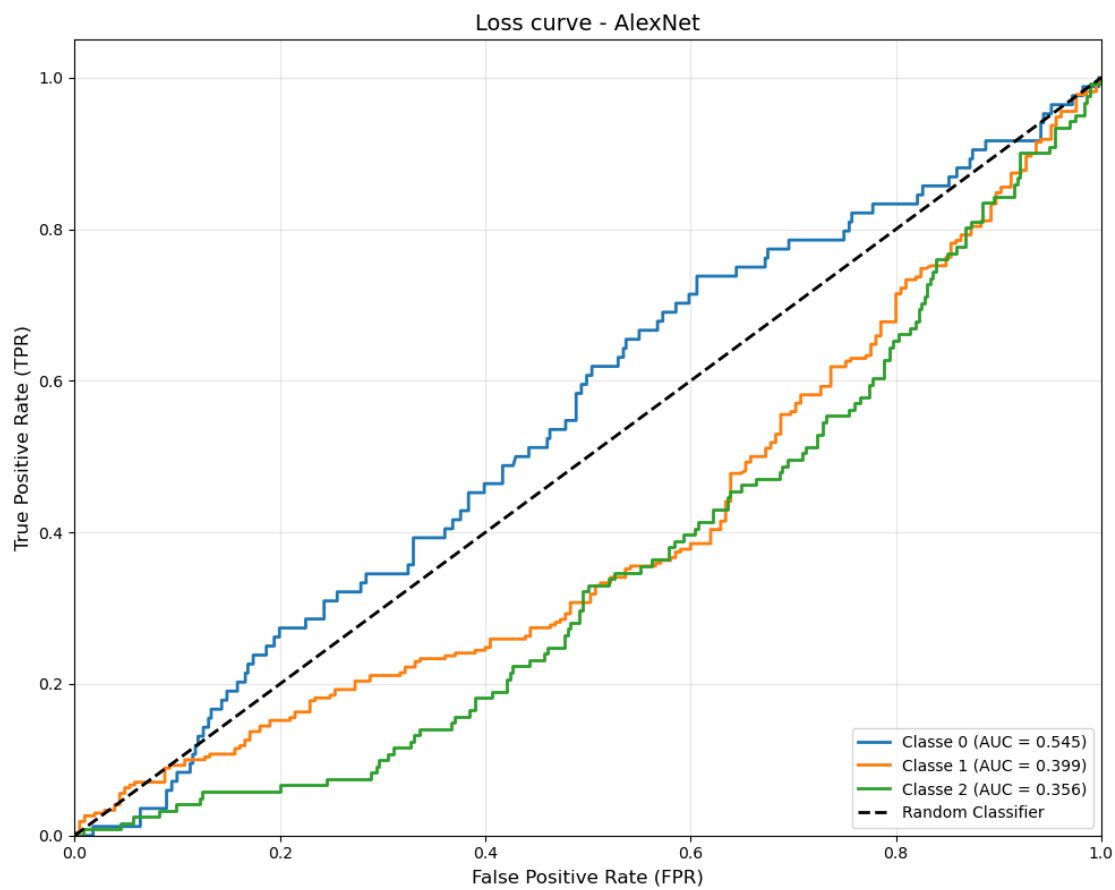


Figura 16: Curve ROC della rete AlexNet.

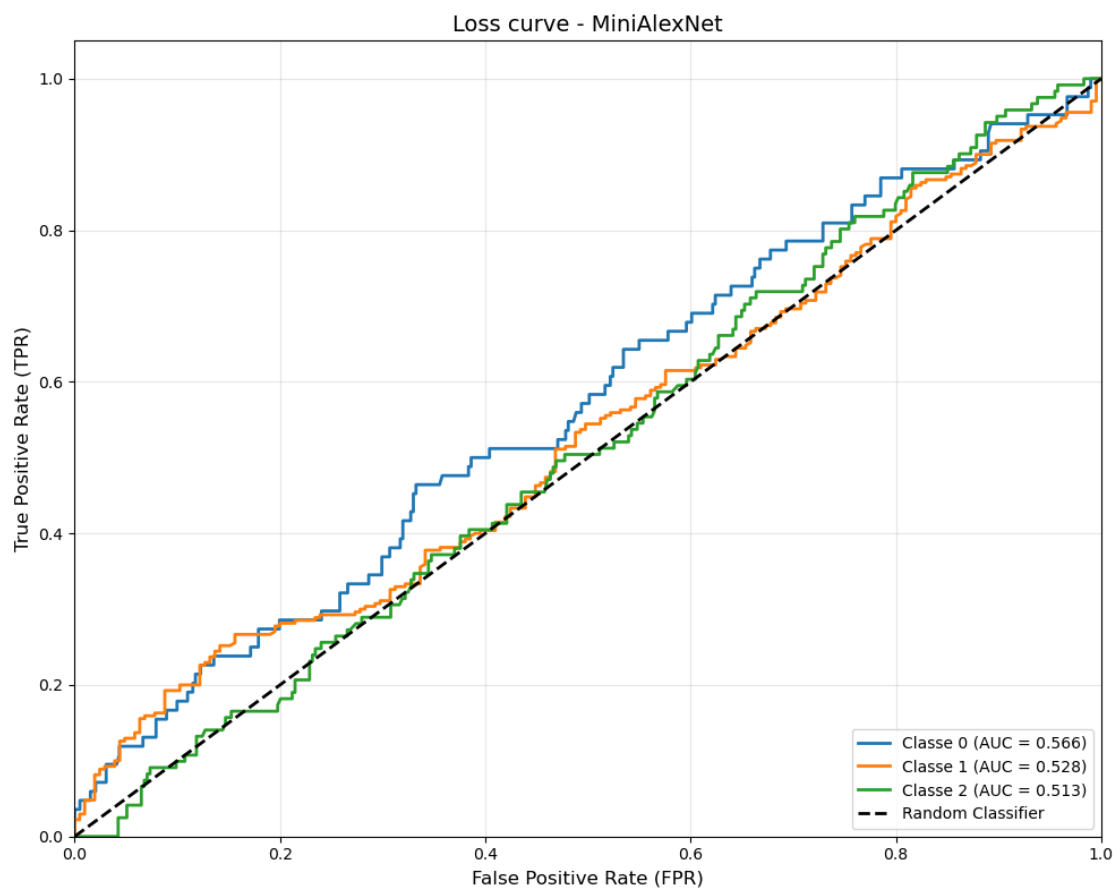


Figura 17: Curve ROC della rete MiniAlexNet.

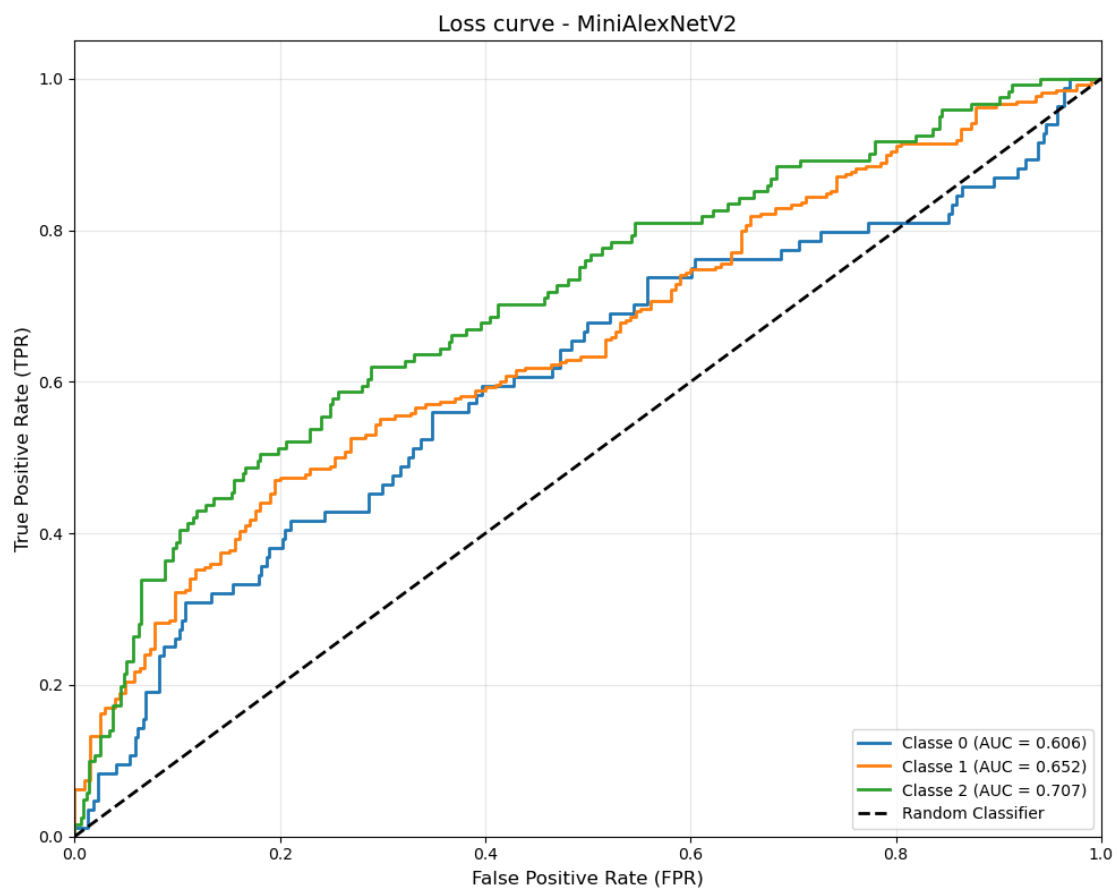


Figura 18: Curve ROC della rete MiniAlexNetV2.